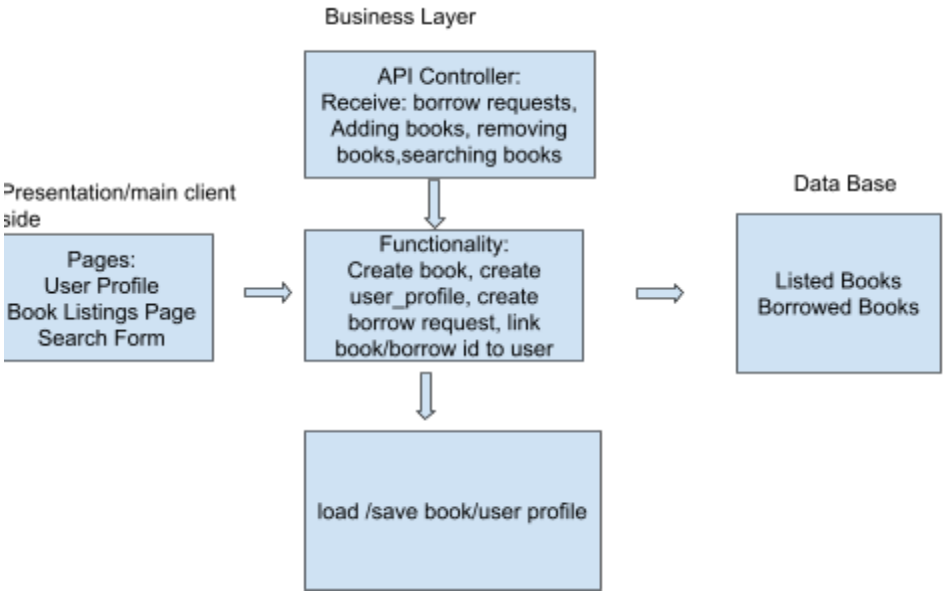
1. Proposed Architecture

The architecture pattern we will be using is Layered. Our book borrowing app involves business calculations and interactions with various databases and AI. So, for the purposes of maintainability, it would be best if we were able to abstract all of those out into different layers. This argument best supports a layered architecture, which is the direction we would like to move in.

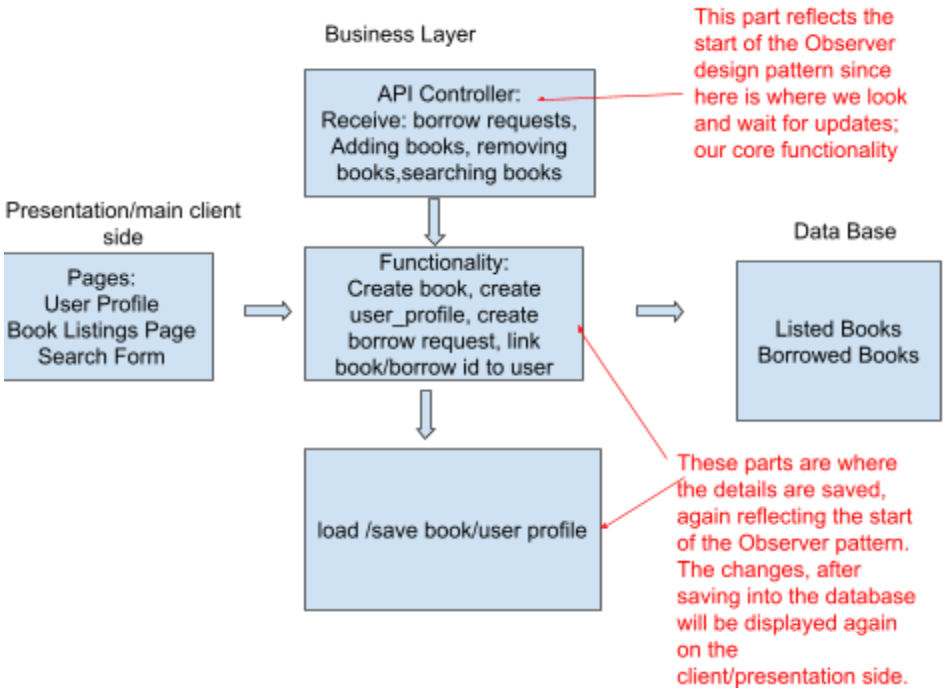
React Native Client -> Transaction Calculator Client -> Service Layer -> Supabase Database

2. High-Level Design Diagram



3. Design Pattern

For our app, the Observer pattern would probably be the most relevant. Our app is based off the ability of users to buy and sell their books and receive live updates on activity that involves them: this is the exact problem that the Observer will be able to solve. This is the core of the product we're building, so we want to ensure this remains a large focus by choosing it as our biggest design pattern. The components that are involved with this design pattern are the User object and borrow request class, as when a borrow request is accepted, then the users affected will be notified and updated. Separating the updating of this information is better than placing all the logic in one large component makes our app scalable for an increase of users, and ensures that every user would receive the same updates.



4. Two Design Decisions

One design decision we made was to use React Native for our frontend instead of standard React. We chose this because we wanted a cross-platform app, prioritizing the ability for users to open it on any mobile or web device. This had a higher barrier to entry than normal React since we had used it for our previous IEs before that point.

Another decision we made was to use Supabase Auth versus Cal Poly sign-ins. We had a team member investigate the process for onboarding an app with Cal Poly sign-ins but eventually determined it would take too long. The approach we will be implementing instead is to verify that emails have the "@calpoly.edu" ending when users sign up for the app and use Supabase Auth again to store passwords securely.